

COMP2019 INTERIM GROUP REPORT

Building an Intelligent Multimodal Search Engine using a Vision Language Model and LanceDB

Date: 12th December 2025

Group: 20

Name	Student ID	UNNC Username
Aariz Sajan	20718606	hcyas4
Karl Munduni	20723175	psykm6
Song	20615047	hfyst11
XU BINDAN	20682988	hcybx1
Zheng	20701628	hcykz1

Supervisor: Dr. Thamil Vaani

COMP2019 INTERIM GROUP REPORT	0
1. Abstract	3
2. Background	3
2.1. Existing Solutions	3
2.1.1. Google Image Search:	3
2.1.2. Shutterstock/getty	5
2.1.3. Pinterest Visual Search	5
2.2. Market Research	6
2.3. Technical Research	7
3. Requirements Specification	8
3.1. Functional Requirements	8
3.2. Non-functional Requirements	10
3.3. Design and Implementation Constraints	11
3.3.1. Technological Constraints	11
3.3.2. Dataset Constraints	12
3.3.3. Hardware & Performance Constraints	13
3.3.4. System Architecture Constraints	13
3.3.5. Development Process Constraints	14
3.3.6. User Interface Constraints	14
3.4. Use Case Diagram	15
3.4.1. Use Case Diagram	15
3.4.2. Use Case: Submit Text Query	15
3.4.3. Use Case: View and Refine Results	16
3.4.4. Use Case: Manage Dataset	16
4. System Design	17
4.1. High Level Architecture Diagram	17
4.2. Module Descriptions	20
4.2.1. Offline Indexing Pipeline	20
4.2.2. Embedding Generation	20
4.2.3. Vector Storage and Indexing Layer	21
4.2.4. Query and Retrieval	22
4.2.5. User Interface	22
4.3. UI Wireframes	23
4.3.1. Low Fidelity	23
4.3.2. Medium Fidelity	25
5. Key Implementation Decisions	26
5.1. Programming Languages and Tools	26
5.2. Reason For Each Decision	26
6. Initial Implementation	28

6.1. Dataset Setup & Preprocessing	28
6.2. Initial Embedding Tests	29
6.3. Initial LanceDB Table Creation	30
6.4. Prototype Query Retrieval	31
6.5. Early UI Prototype	32
7. Problems Encountered	33
7.1. Technical Problems	33
7.1.1. Technical Problems With System 1	33
7.1.2. Technical Problems With System 2	34
7.1.3. Technical Problems With UI/UX and Streamlit WebApp	37
7.2. Management Problems	39
8. Timeline	41
8.1. Gantt Chart	41
8.2. Planned Events Timeline	41
8.2.1. System 1 (Offline Indexing):	41
8.2.2. System 2 (Query and retrieval):	42
8.2.3. System 3 (UI/UX, System Design, Quality Eval)	43
9. Meeting Minutes	43
11. References	45

1. Abstract

Current ways of searching for images are limited in such a way that it is very hard to find a specific photo in a large digital pile. This is because regular search systems rely on simple labels like "dog" or "ball" which means the full story of the image is missed. So to combat this, we are dedicating this project into building an intelligent search engine that aims to bridge this "semantic gap". The system will leverage a pre-trained Visual Language Model that is trained to both understand pictures and the user's natural language queries that will then turn both the picture and the query into vector embeddings. These embeddings will be stored and accessed from a database so that when you search, the system matches the embeddings of your search to the embeddings of the image. This means it finds images based on true meaning and context.

2. Background

2.1. Existing Solutions

The following include some similar existing solutions to what this project is attempting to achieve and their positives and negatives.

2.1.1. Google Image Search:

Google Image Search works by crawling the web for images using the tag, indexing them based on both the image file and the context in which they appear, and then ranking them using a combination of metadata, page relevance, and AI-based visual analysis.

When a user performs a search, Google matches the query with images using semantic understanding, metadata, and visual similarity, while also applying SafeSearch filters if needed.

The final results are ranked using relevance, image quality, page authority, loading performance, and user engagement signals, with Google sometimes enhancing listings using rich result badges where structured data is present.

Pros:

Combining metadata with AI-based visual understanding allows Google to identify objects, scenes, text, and concepts even when metadata is poor. Google considers the landing page, not just the image, improving relevance and preventing low-quality or misleading images from ranking.

Cons:

If a site uses poor markup (e.g., CSS background images, missing alt text), images may never be indexed even if they are useful. AI can misinterpret complex, abstract, or ambiguous images, especially without strong context. High-quality images on slow or low-quality pages may rank poorly because the page reduces trust or usability.

2.1.2. Shutterstock/getty

Shutterstock's image search works by combining keyword-based search and AI-driven visual similarity search. The system tags images using contributor keywords and automated machine vision, then ranks them based on relevance, popularity, and user behavior.

Pros:

Tags act as pre-processed labels so the search engine doesn't need to analyze every image from scratch. Tags supports advanced search filtering.

Cons:

Reliance on contributor's tagging: poor tagging means an image may not be found. Sometimes critical visual details aren't included in the tags. An image tagged as "happiness" by one contributor might be tagged "family" or "vacation" by another. This inconsistency affects search relevance.

2.1.3. Pinterest Visual Search

Pinterest lets you enter search terms / keywords into its main search bar. The system then returns Pins (images) whose associated metadata (titles, descriptions, board names, maybe page URLs) and computed semantics match the query. Internally, Pinterest uses a

unified embedding + ranking system that combines text-based metadata (title, description, board, associated text) with image-content signals and engagement data to determine relevance of Pins for a given query.

Pros:

Pinterest's use of metadata, embeddings and inferred annotations means search can return items that match conceptually, even if exact keywords differ and this is good for broader searches. By combining text, visual similarity (when relevant), and engagement/popularity signals, search tends to surface content that's both relevant and popular (or high-quality).

Cons:

If a pin has poor, missing or uninformative title or description or inaccurate/misleading metadata it may not surface even if the image matches the query. For broad or vague queries, you may get mismatched results, because the embedding, metadata and popularity ranking may surface loosely related content

2.2. Market Research

According to the semantic search market research report 2033, the Global Semantic Search market size was valued at \$6.2 billion in 2024, and is forecasted to hit \$32.5 billion by 2033. This is fueled by the rapid adoption of AI and natural language processing technologies as well as the fact that organisations are continually finding ways to enhance the accuracy and relevance of information retrieval. Healthcare, the financial and insurance industries and e-commerce have recently made greater use of semantic search in things like improving recommendation engines, medical literature retrieval and fraud detection. Our project aims to explore opportunities in the semantic search field.

2.3. Technical Research

This project will use a pre-trained vision language model (VLM). As for the model, we have chosen openAi's CLIP. CLIP (Contrastive Language-Image Pre-training), has

revolutionised the way machines understand the relationship between images and text. These models can map both modalities into a shared embedding space, where semantically similar concepts (e.g., a picture of a cat and the text "a photo of a cat") are located close to each other. CLIP will be used to generate vector embeddings that represent the semantic content of both images and text queries.

Next, these multimodal embeddings will need to be stored, indexed, and queried. To operationalise this capability for search, a specialised database is required to store and perform similarity searches on these high-dimensional vector embeddings and so LanceDB was chosen. LanceDB is an open-source, embedded vector database that is particularly well-suited for this task due to its high performance, zero-copy data access, and ease of integration into Python-based AI applications, eliminating the need for complex database server management.

3. Requirements Specification

3.1. Functional Requirements

Requirement	Description
<u>Image Data Ingestion</u>	• The system shall allow an admin to upload or import image datasets. • The system shall validate image formats. • The system shall store metadata such as filename, dimensions, and upload date.
<u>Image Preprocessing</u>	• The system shall uniformly resize and normalise all images before embedding. • The system shall batch-process images for efficient embedding computation. •
<u>Embedding Generation</u>	• The system shall generate multimodal embeddings using the CLIP model for each image. • The system shall generate a text embedding for each user query. • The system shall store embeddings in LanceDB.
<u>Index Construction & Storage</u>	• The system shall build a vector index using LanceDB for fast similarity search. • The system shall update the index automatically

	when new images are added.
<u>Search Query Handling</u>	• The system shall allow the user to enter a natural-language text query. • The system shall convert the query into a vector representation. • The system shall perform vector similarity search using the LanceDB index.
<u>Retrieval & Ranking</u>	• The system shall return a ranked list of top-K similar images based on cosine similarity. • The system shall display similarity scores for each retrieved result.
<u>User Interface (Web App)</u>	• The system shall provide a web interface to enter queries and view search results. • The system shall allow users to preview images in a gallery layout. • The system shall provide pagination or infinite scroll for large result sets.
<u>Performance Logging</u>	• The system shall log query latency, similarity scores, and retrieval times. • The system shall store performance metrics for evaluation.
<u>Admin Operations</u>	• The system shall allow admin users to trigger reindexing. • The system shall allow admin users to upload their own images for indexing • The system shall allow admins to view dataset statistics (number of images, missing metadata, etc.)

3.2. Non-functional Requirements

<u>Performance</u>	• The system shall return search results within ≤ 2 seconds for a query on a dataset of 10k+ images. • The embedding process shall support batch processing of at least 100 images/min.
<u>Accuracy & Quality</u>	• The similarity search shall maintain high semantic relevance, evaluated through Precision@K. • The CLIP model must generate consistent embeddings for identical inputs.
<u>Scalability</u>	• The vector database shall support scaling to millions of image embeddings without significant performance degradation. • The indexing pipeline shall accommodate incremental dataset updates.
<u>Usability</u>	• The user interface shall be intuitive and require no training. • All user-visible components shall follow consistent layout and colour standards.
<u>Reliability</u>	• The system shall handle invalid inputs gracefully (e.g., empty queries, corrupted images). • The system shall guarantee no loss of embeddings or metadata during indexing.
<u>Security</u>	• Only authenticated admin users shall be able to upload images or rebuild indexes. • The system shall not expose internal file paths, embeddings, or system logs to end users.
<u>Maintainability</u>	• The system shall modularise code into indexing, embedding, and search subsystems for easy maintenance. • The system shall include clear documentation for all modules.
<u>Compatibility</u>	• The system shall run on Python 3.10+ and be deployable on Windows, macOS, or Linux. • The system shall integrate with CLIP and LanceDB without requiring proprietary services.
<u>Ethical & Dataset Constraints</u>	• Only publicly available datasets (Unsplash Lite, COCO, Flickr30k) shall be used, in accordance with ethical guidelines.

	• No personal or copyrighted user-owned images shall be stored or processed.
--	--

3.3. Design and Implementation Constraints

This section outlines the constraints that limit or influence the design and development of the proposed multimodal search engine. These constraints arise from the client's requirements, available resources, chosen technologies, and the scope of the project.

3.3.1. Technological Constraints

Mandatory use of a Vision–Language Model (VLM):

The client requires semantic search capabilities, therefore the system must utilise OpenAI's CLIP model for generating image and text embeddings. No alternative embedding models are permitted within the project scope.

Vector Database Requirement:

The system must store embeddings in LanceDB, as it was client requested and is optimised for similarity search, lightweight, and allows local development without external hosting.

Python Ecosystem:

All backend pipelines—embedding generation, indexing, and retrieval—must be implemented in Python due to library support (PyTorch, LanceDB) and group competency.

No Cloud Infrastructure:

All processing must be done locally. Cloud services (AWS, GCP, Azure) are not allowed due to ethical approval, cost limitations, and deployment constraints.

3.3.2. Dataset Constraints

Use of Public & Open-Source Image Datasets Only:

Only openly licensed datasets such as Flickr30k, Unsplash Lite, or COCO may be used. Copyrighted or user-uploaded datasets cannot be indexed due to ethical restrictions.

Dataset Size Limitations:

The system must operate on a dataset size feasible for local indexing and retrieval within the timeframe. This restricts the number of embeddings the system can reasonably process.

3.3.3. Hardware & Performance Constraints**Limited Computational Resources:**

The system must run on standard student laptops without dedicated GPUs, which restricts:

- Batch size for embedding generation
- Real-time inference speed
- Maximum dataset size for the prototype

As a result, a unified computational limit must be standardized.

Local Storage Limitations:

LanceDB runs locally, therefore the database size must remain small enough to be stored and queried within typical laptop storage ($\leq 10\text{--}15$ GB).

3.3.4. System Architecture Constraints**Two-Subsystem Architecture:**

System 1 (Indexing): offline embedding generation + database creation

System 2 (Retrieval): online querying + similarity search

This separation is mandatory to meet the project's sprint structure and responsibilities assigned to group members.

3.3.5. Development Process Constraints**Scrum Methodology Requirement:**

The development process must follow the University-mandated Scrum methodology, including sprints, sprint reviews, and a working increment at each stage.

Time Constraints:

The interim prototype must be delivered within the first half of the semester; therefore, the design must prioritise features that can be completed within short sprints.

3.3.6. User Interface Constraints**Web-Based Prototyping Only:**

The UI must be implemented using lightweight frameworks such as Streamlit or Flask, as mobile app development falls outside the project scope.

Minimalist Interface:

The UI must remain simple, functional, and suitable for testing retrieval functionality. Advanced UI/UX design is not required for this stage.

3.4. Use Case Diagram**3.4.1. Use Case Diagram**

The actual Use Case Diagram is provided in **Appendix B**.

3.4.2. Use Case: Submit Text Query**Actors:**

- User (primary)
- Vector Database — LanceDB (supporting)

Description:

- User enters a text query into the search interface.
- System validates the query format.
- System generates a text embedding using the CLIP text encoder.
- System sends the embedding to the Vector Database (LanceDB).
- Vector Database performs similarity search and returns top-k results.
- System receives the ranked results.

- System displays the results to the User.

3.4.3. Use Case: View and Refine Results

Actors:

- User (primary)

Description:

- User views the displayed search results.
User applies filters (e.g., relevance score, timestamp).
- System updates the list of results according to the selected filters.
- User selects an item to inspect details.
- System displays the detailed metadata and image.

3.4.4. Use Case: Manage Dataset

Actors:

- Admin (primary)

Description

- Admin opens the dataset management interface.
- Admin uploads, removes, or updates dataset items.
- System validates the changes.
- System triggers re-indexing of the modified dataset.
- System stores updated embeddings in the Vector Database.

4. System Design

4.1. High Level Architecture Diagram

The overall system has been designed as a two-part multimodal retrieval pipeline, consisting of an offline indexing subsystem and an online query-and-retrieval subsystem. This division ensures that the computationally intensive stages of the workflow,

particularly embedding generation and index construction, are completed prior to user interaction, thereby enabling fast and responsive search performance at runtime.

The high-level architecture is illustrated in **Appendix C**, which summarises the data flow between the major components of the system.

As seen in **Appendix C**, the system has two main parts: an offline indexing subsystem and an online query-and-retrieval subsystem.

The offline indexing subsystem processes the image dataset for retrieval. Raw images are acquired, preprocessed for consistency, and then passed through a Vision-Language Model (e.g., CLIP) to generate high-dimensional semantic embeddings. These embeddings and their metadata are stored and indexed in LanceDB. This computationally intensive, offline process ensures fast searchability later.

The query-and-retrieval subsystem operates in real time, responding to user queries. A natural-language query is encoded using the same Vision-Language Model, placing it in a shared semantic embedding space with the image vectors. This query vector is passed to LanceDB, which performs a similarity search (e.g., IVF-PQ) over the indexed embeddings. The system returns the most semantically relevant images, ranked by score, providing highly responsive performance because the heavy lifting was done offline.

The architecture in Figure 4.1 reflects modern design principles widely adopted in large-scale multimodal search systems. By separating offline and online responsibilities, the system ensures scalability, efficient retrieval, and the flexibility to replace or upgrade individual components (such as the embedding model or the indexing strategy) without requiring significant changes to the rest of the system.

4.2. Module Descriptions

4.2.1. Offline Indexing Pipeline

The offline indexing pipeline is responsible for preparing the image dataset so that it can be efficiently searched at runtime. This pipeline begins with the ingestion of raw images from the selected dataset and the organisation of these files into a structure that facilitates deterministic processing. Each image is then passed through a sequence of preprocessing operations, including resizing, normalisation, and format standardisation, to ensure that all images conform to the input requirements of the Vision–Language Model used in later stages. Once preprocessing is completed, the images are batched to optimise throughput during embedding generation. The indexing pipeline operates independently of the query system and can be executed repeatedly when new data becomes available. Its output is a collection of preprocessed images ready for semantic embedding.

4.2.2. Embedding Generation

The embedding generation engine is the core analytical component responsible for converting both images and text into a shared semantic embedding space. The system employs a state-of-the-art Vision–Language Model, such as CLIP, which contains two encoders: one for processing images and another for processing textual descriptions. During the offline stage, each image is passed through the model’s vision encoder to produce a high-dimensional vector representation that encapsulates its semantic content. These embeddings reflect relationships that are not limited to superficial features but instead capture deeper contextual meaning. By using the same model to encode text queries during the online phase, both modalities are mapped into a space where semantic similarity can be measured directly. The embedding engine therefore plays a critical role in ensuring that the search results align with the user’s intended meaning rather than mere keyword matches.

4.2.3. Vector Storage and Indexing Layer

The vector storage and indexing layer provides the underlying data infrastructure that makes semantic search computationally feasible. Once embeddings have been generated, they are inserted into a LanceDB table together with relevant metadata such

as file paths, labels, and any associated descriptive text. The database then constructs a vector index using an approximate nearest-neighbour method (such as IVF-PQ) to enable rapid similarity search across potentially large numbers of embedding vectors. This layer ensures that queries can be processed with low latency even when the dataset grows substantially. The separation between storage and indexing further allows the system to update the index incrementally or reconstruct it entirely without disrupting the rest of the architecture. The resulting indexed database forms the central repository against which all user queries are evaluated.

4.2.4. Query and Retrieval

The query and retrieval engine is responsible for processing user queries at runtime and returning the most relevant images based on semantic similarity. When a user submits a natural-language query, it is first encoded by the text component of the Vision–Language Model, producing a vector that resides in the same embedding space as the precomputed image representations. This query vector is then passed to the LanceDB index, which performs a similarity search to identify the embeddings that are closest in semantic space. The system ranks the results according to similarity score, ensuring that the most contextually appropriate images are returned first. The retrieval engine therefore acts as the bridge between user intent and the stored dataset, transforming textual descriptions into meaningful visual results with minimal delay.

4.2.5. User Interface

The user interface layer provides the interaction point through which users access the system. The interface is designed to be simple and intuitive, allowing users to enter natural-language search queries and immediately view the results returned by the retrieval engine. The UI displays images in a structured format and may also present accompanying metadata to provide additional context. Although the visual appearance of the interface is not the primary focus at this stage of the project, the prototype demonstrates the essential functionality needed to validate the underlying system. By separating the UI from the core retrieval logic, the design allows for future enhancements,

including image-based queries, filtering options, and improved visual presentation, without requiring changes to the underlying modules.

4.3. UI Wireframes

To support the design of the user interface, low- and medium-fidelity wireframes were produced to explore layout, interaction flow, and result presentation. The wireframes focus on providing a minimal and intuitive search experience that enables users to submit natural-language queries and inspect retrieved images efficiently.

These wireframes serve as design artefacts to validate usability assumptions rather than final interface designs. The complete set of UI wireframes and prototype screenshots is provided in **Appendix A**.

5. Key Implementation Decisions

5.1. Programming Languages and Tools

Choosing the correct programming languages and tools is a critical step in developing any software engineering project, especially one involving machine learning and multimodal retrieval. Selecting the appropriate programming language, frameworks, and development environment ensures that the system can be implemented efficiently and maintained reliably throughout the project lifecycle. The tools chosen directly influence development speed, system reliability, performance, and the feasibility of implementing advanced features such as image-text embeddings and vector similarity search.

After careful consideration of the project requirements, which involve integrating a Vision-Language Model with a vector database and deploying a lightweight user interface, the chosen programming language is Python. The main tools used in this project include HuggingFace Transformers, LanceDB, and Streamlit UI.

5.2. Reason For Each Decision

Python was selected as the main programming language because it is one of the most compatible and widely-used languages in the field of machine learning. Its concise syntax improves readability and reduces development time, while its extensive ecosystem of libraries (such as Pandas, TensorFlow, and PyTorch) supports a wide range of machine learning and data processing tasks. Python also enables seamless integration between the Vision-Language Model, the vector database, and the UI layer, which makes it well-suited for end-to-end development.

HuggingFace Transformers was chosen as the core framework for model integration. It allows the project to utilize CLIP, a state-of-the-art Vision-Language Model developed by OpenAI, without needing to train a model from scratch. Since CLIP is pretrained on large-scale datasets, it provides high-quality and reliable embeddings for both images and text. HuggingFace ensures standardised preprocessing, consistent performance, and easy GPU acceleration, all of which are essential for generating embeddings efficiently.

LanceDB was selected as the vector database for several reasons. Primarily, LanceDB was the database that was requested from our client (Wysetime) for its specialisation in similarity search using high-dimensional vectors. As addition to that, LanceDB is lightweight compared to other full-scale vector database servers, making it ideal for this academic project. LanceDB offers fast retrieval speed, low latency, and efficient indexing mechanisms, which supports scalability as the dataset grows. Its Python integration also allows smooth communication between the embedding pipeline and the search module.

Finally, Streamlit was chosen as the user interface framework because it works seamlessly with Python functions and machine learning workflows. It eliminates the need for complex front-end development, significantly reducing the time required to build an interactive UI. Streamlit enables rapid prototyping, easy iteration, and provides built-in components for displaying images and handling text input, making it ideal for demonstrating the system's functionality.

6. Initial Implementation

6.1. Dataset Setup & Preprocessing

The first step we tackled was getting our image dataset ready for processing. We chose to work with the Unsplash Lite dataset because it's openly licensed and contains a good variety of high-quality images that would test our system properly.

Setting up the dataset wasn't as straightforward as we initially thought. We had to deal with a few issues when we first started. Some of the image files were corrupted, and others had inconsistent formats (mixing JPEGs and PNGs), and the folder structure needed organising before we could even start preprocessing. We spent a fair bit of time cleaning this up, removing corrupted files, and creating a standardised directory structure that would make batch processing easier later on.

For preprocessing, we needed to make sure every image met CLIP's input requirements. This meant resizing all images to 224×224 pixels and normalising the pixel values to match what CLIP expects during processing. We wrote a Python script using PIL (Python Imaging Library) to handle the preprocessing pipeline, which could process images in batches to speed things up. The script also logged any images that failed preprocessing so we could investigate them separately.

By the end of this stage, we had a clean, preprocessed dataset ready for embedding generation, with all images stored in a consistent format and organised structure.

6.2. Initial Embedding Tests

Once we had our preprocessed images ready, we ran our first round of embedding tests to make sure CLIP was actually working as expected. We selected a small batch of around 50 images from different categories (animals, landscapes, objects, people) to see if the embeddings made sense.

The goal was to verify that CLIP could generate 512-dimensional vectors for each image and that these vectors actually captured semantic meaning. We loaded the CLIP model using HuggingFace Transformers and passed our test images through the vision encoder.

The results were promising. When we looked at the embedding space, images that were visually or conceptually similar (like different photos of cats, or various sunset scenes) had embeddings that were much closer together compared to completely unrelated images. We calculated cosine similarity between a few pairs of embeddings manually to confirm this, and the numbers backed up what we were seeing.

However, we did notice that the embedding generation was quite slow on our laptops without dedicated GPUs. Processing even just 50 images took longer than we'd like. This made it clear that we'd need to implement proper batching and possibly look into optimising the inference process for when we scaled up to the full dataset.

Overall though, these initial tests confirmed that CLIP was producing meaningful embeddings, which gave us confidence to move forward with building out the full indexing pipeline.

6.3. Initial LanceDB Table Creation

After generating our first batch of image embeddings, we created an initial LanceDB table to test how the database would store and organise the vectors. We started by setting up a local LanceDB instance and making a new table with three basic fields:

1. 512-dimensional vector for the CLIP embedding
2. The image filename or file path
3. Some simple metadata such as image size or labels

6.4. Prototype Query Retrieval

The prototype query retrieval system was built to test whether a text query could return similar images from the LanceDB table. When the user enters a short sentence, the system converts it into a **512-dimensional CLIP text embedding**, which matches the format of the stored image embeddings.

The system successfully returned the top-K images that were closest to the query in embedding space. For basic queries such as "a cat", "sunset", or "a person riding a bike", the results were mostly accurate and showed that the embeddings were aligned correctly.

Some limitations were:

- very short queries gave broad or vague results
- brute-force search became slower as more images were added
- returned file paths had to be converted to displayable images for the UI

Even with these issues, the prototype retrieval was a success because it demonstrated that CLIP text encoding + LanceDB similarity search worked correctly. This gave us a solid base to add indexing (IVF-PQ) and connect the system to the UI in later stages.

6.5. Early UI Prototype

This section shows the initial browser implementation of the planned User Interface.

This initial version was built using streamlit, the standard python library for User Interface builds. In addition, the results that it generated were purely sample data from the MockSearchAPI. There was no logic or functional query retrieval during the time of the initial UI implementation. However, it does clearly display our intentions with regard to UI.

With that said, we will be getting client feedback on the design so that we may improve it and potentially add use-case relevant features which will better the user experience of the client and their team.

An early user interface prototype was developed using Streamlit to demonstrate the intended interaction flow and result presentation. At this stage, the interface was connected to mock search data rather than the full retrieval pipeline, as the focus was on validating layout and usability assumptions.

Screenshots of the prototype interface are included in **Appendix A**, alongside the corresponding wireframes.

7. Problems Encountered

7.1. Technical Problems

7.1.1. Technical Problems With System 1

1. Image Dataset Quality and Preprocessing

A key challenge in System 1 is ensuring that publicly sourced images are consistently suitable for embedding generation. Images often vary in resolution, aspect ratio, and visual quality, and some may be weakly relevant or duplicated.

This affects system reliability because inconsistent preprocessing or improper normalisation can distort visual features, leading to embeddings that do not accurately represent the original image content.

2. Image Embedding Generation at Scale

System 1 must generate embeddings for a large number of images using the CLIP image encoder, which is computationally demanding. Efficient batching and resource management are required to prevent performance bottlenecks.

Failures during long embedding runs or mismatches between images and embeddings can result in missing or incorrect vectors, negatively impacting retrieval accuracy in later stages.

3. Indexing and Consistency in LanceDB

Once embeddings are generated, they must be correctly indexed in LanceDB. Maintaining consistency between stored embeddings and image metadata is non-trivial, especially when reindexing or scaling the dataset.

Small indexing errors, such as missing vectors or misaligned metadata, can significantly degrade search performance, even when query embeddings are correctly generated.

7.1.2. Technical Problems With System 2

1. Converting Natural Language Queries into High-Quality Embeddings

A major challenge lies in ensuring that natural language queries provided by users are consistently transformed into meaningful, discriminative vector embeddings using the CLIP text encoder. Natural language is inherently ambiguous, and small variations in phrasing can produce embeddings with significantly different semantic distances.

This affects retrieval quality because:

- CLIP may interpret certain phrases differently than intended.
- Queries with multiple concepts (e.g., "a man standing on a beach at sunset holding a surfboard") require CLIP to correctly capture the dominant semantic features.
- Short queries ("cat", "car", "red") tend to be under-described, making embedding quality inconsistent.

2. Tuning IVF-PQ Index Parameters in LanceDB

LanceDB's IVF-PQ indexing scheme enables fast approximate nearest neighbour search, but selecting optimal parameters requires significant trial-and-error.

Key parameters include:

- nlist (number of coarse clusters)
- nprobe (number of clusters searched per query)
- PQ bits and sub-vector sizes
- The difficulty arises because:
 - Increasing accuracy slows down retrieval
 - Increasing speed often degrades result quality

Finding the right parameters to balance accuracy and speed requires repeated reindexing.

3. Integration of Encoding, Querying, and Ranking Components

Even with good embeddings and a well-configured index, integrating the individual components of System 2 into a smooth, cohesive pipeline is non-trivial.

The system must:

1. Accept a user query (text or image)
2. Encode it using the correct CLIP encoder
3. Query LanceDB for nearest neighbours
4. Rank the results by similarity score
5. Return outputs in a format suitable for the UI

The main difficulties encountered include:

- Ensuring that all components use the same embedding dimensions and preprocessing steps.
- Handling differences between image-to-text and text-to-image similarity spaces.
- Maintaining consistent performance across different types of queries.
- Preventing bottlenecks where one stage (e.g., encoding) becomes significantly slower than others.
- Designing the pipeline so that it supports future UI improvements and asynchronous queries without breaking.

Small misalignments (e.g., mismatched tensor shapes, floating-point precision differences, incorrect normalisation steps) can cause large performance drops or incorrect retrieval results.

7.1.3. Technical Problems With UI/UX and Streamlit WebApp

1. Limited Customisation for Interactive Components

Streamlit is primarily designed for dashboards and data-driven applications. As a result, it provides only basic layout and design controls. This constrained the team's ability to implement more sophisticated interface elements required for our application, such as:

- dynamic image galleries
- interactive result visualisation
- custom animations or transitions
- responsive multi-panel layouts

Streamlit's layout system often conflicted with the desired structure, leading to visually inconsistent or cluttered screens.

2. Lack of Full Front-End Control

Streamlit abstracts most of the front-end logic away, making it difficult to directly control:

- DOM elements
- CSS behaviour
- precise positioning
- custom JavaScript interactions
- 3D or canvas-based UI features

Features such as real-time search response indicators, draggable elements, or visual similarity maps (e.g., 2D/3D embedding visualisations) were effectively impossible without heavy workarounds.

3. Poor Separation of Front-End and Back-End Concerns

Streamlit's architecture tightly couples the UI and backend logic. For our system—where System 1 (Indexing) and System 2 (Retrieval) should live separately—this coupling created several friction points:

- inability to cleanly develop the front-end independently
- difficulty mocking backend responses
- reduced flexibility when testing different retrieval interfaces

A decoupled JS front-end is more suitable for this architecture.

4. Difficulty Implementing a Modern, Professional UI

The client indicated that the final product should be visually modern and professionally presented. Achieving this aesthetic with Streamlit is extremely difficult because:

- the design system is rigid
- CSS overrides are unreliable
custom theming options are limited
- components do not support complex styling

The UI produced with Streamlit appeared more like a functional prototype rather than a polished product.

7.2. Management Problems

The group encountered several management-related challenges during the early stages of the project. Communication proved to be a significant difficulty, as two group members were non-native English speakers. This created misunderstandings during discussions, slowed decision-making, and required additional clarification when distributing tasks or reviewing technical concepts. These language barriers also contributed to uneven comprehension of project requirements, which in turn affected the group's ability to progress at a consistent pace.

Another issue involved maintaining motivation and work ethic across the team. While some members were highly proactive, others struggled to sustain engagement with the workload or self-direct their tasks effectively. This inconsistency made it difficult to coordinate efforts, ensure deadlines were met, and maintain momentum throughout the sprints. Furthermore, differences in how quickly members understood the material - particularly concepts related to vector databases, multimodal embeddings, and system decomposition - led to additional delays and required repeated explanations to ensure alignment before implementation work could begin.

These problems were mitigated through structured guidance from the client, whose interventions significantly improved team organisation and morale. The client encouraged the group to adopt clearer planning through the creation of a Gantt chart, which provided a shared visual timeline and improved overall coordination. He also recommended working in parallel rather than strictly sequential phases, enabling different subsystems to be developed simultaneously and reducing bottlenecks caused by uneven understanding or pacing. Moreover, the client fostered a more positive and motivated working environment by emphasising the long-term value of the project, reminding the group that a well-executed multimodal search engine would be a strong addition to their professional portfolios and future CVs. This combination of structured planning, parallelism, and motivational support helped stabilise the team's workflow and improve overall collaboration.

8. Timeline

8.1. Gantt Chart

The Gantt Chart is provided in **Appendix D**.

8.2. Planned Events Timeline

8.2.1. System 1 (Offline Indexing):

21/11/2025 - 25/11/2025: Collect images, clean corrupted files, organise folder structure

26/11/2025 - 28/11/2025: Resize, normalize, batch images for CLIP

29/11/2025 - 12/12/2025: Developing the Embedding System

13/12/2025 - 19/12/2025: Embedding generation

20/12/2025 - 26/12/2025: Metadata construction

27/12/2025 - 31/12/2025: Embedding storage

01/01/2026 - 08/01/2026: Index construction

09/01/2026 - 01/04/2026: Evaluation, fix bugs, speed improvement, accuracy improvements

8.2.2. System 2 (Query and retrieval):

20/11/2025 - 25/11/2025: setup&data

25/11/2025 - 30/11/2025: feature extraction

01/12/2025 - 10/12/2025: indexing (IVP-FQ)

11/12/2025 - 20/12/2025: query implementation

21/12/2025 - 20/01/2026: Text Query Encoding - Use the text encoder of CLIP to obtain a 512-dimensional vector for the user's one sentence

21/01/2026 - 20/02/2026: Similarity Calculation and Retrieval - Perform approximate retrieval using FAISS to obtain top-K; calculate the actual cosine score, then sort and return top-K.

21/02/2026 - 20/03/2026: Post-processing of Results and Formatting of Return

21/03/2026: Evaluation

8.2.3. System 3 (UI/UX, System Design, Quality Eval)

20/11/2025 - 26/11/2025: System architecture diagrams, data contracts, interfaces

27/11/2025 - 05/12/2025: UI skeleton (Streamlit layout)

06/12/2025 - 12/12/2025: Mock integration, fake search results, API structure

13/12/2025 - 20/12/2025: Final API definitions + UI improvements

21/12/2025 - 15/01/2026: UI polishing + integration preparation

16/01/2026 - 10/02/2026: Integration Round 1 (backend + text encoder)

11/02/2026 - 20/02/2026: Integration Round 2 (real embeddings + LanceDB)

21/02/2026 - 10/03/2026: UI enhancements + result formatting

11/03/2026 - 20/03/2026: System testing + bug fixing

21/03/2026 - 31/03/2026: Final polishing, documentation, presentation prep

9. Appendices and Meeting Minutes

9.1. Appendix A: User Interface

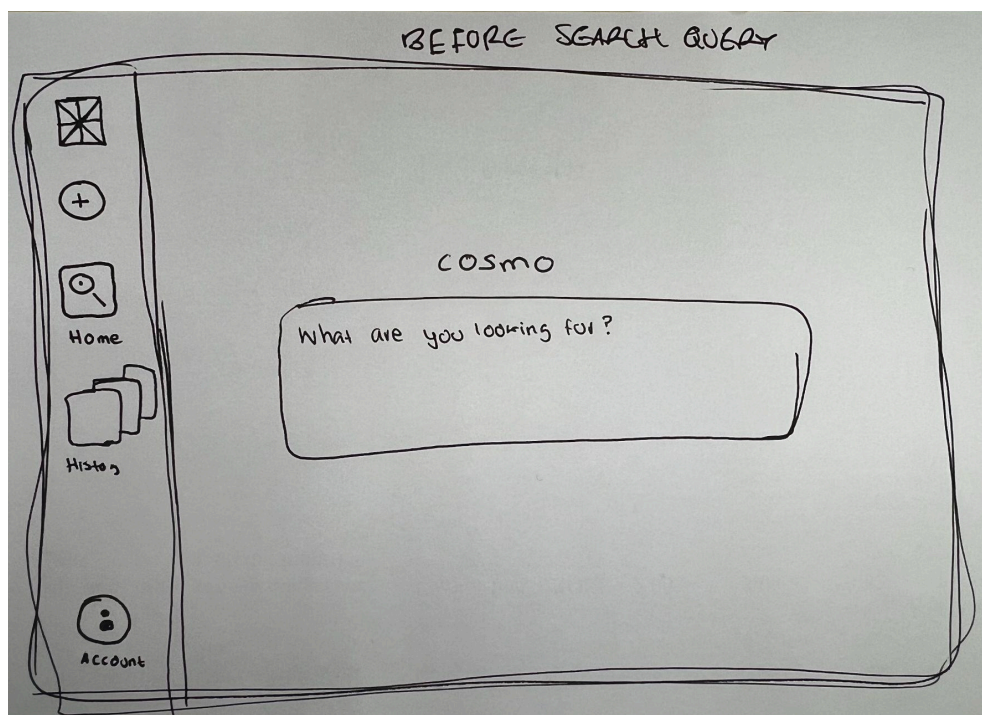


Figure A.1: Low Fidelity Wireframe of Before Query User Interface

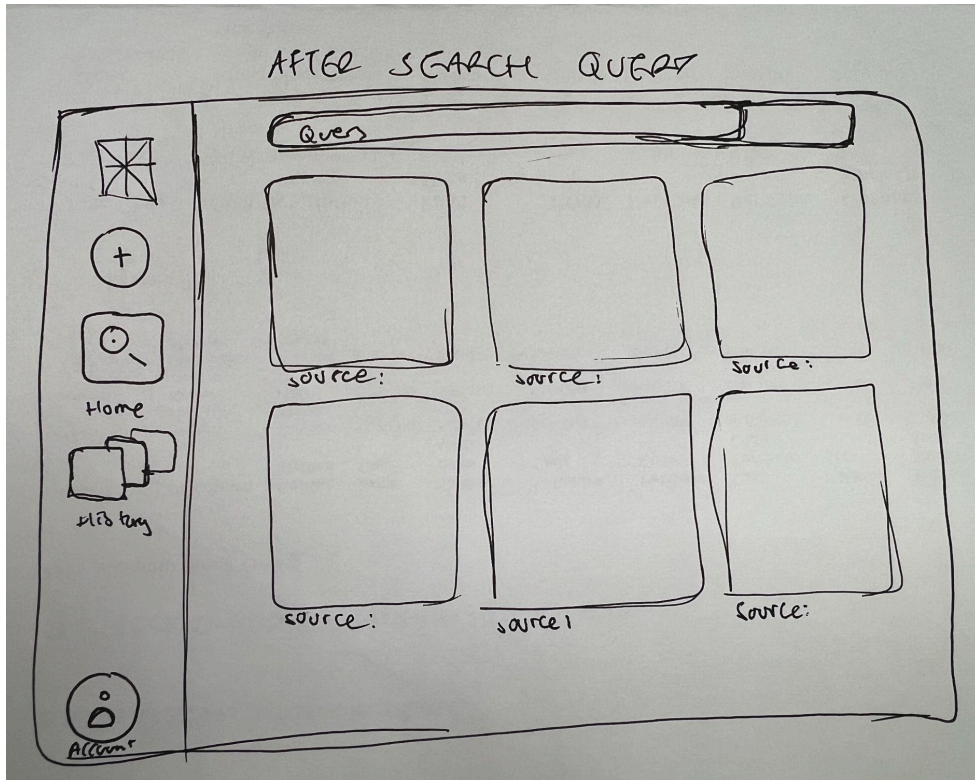


Figure A.2: Low Fidelity Wireframe of After Query User Interface

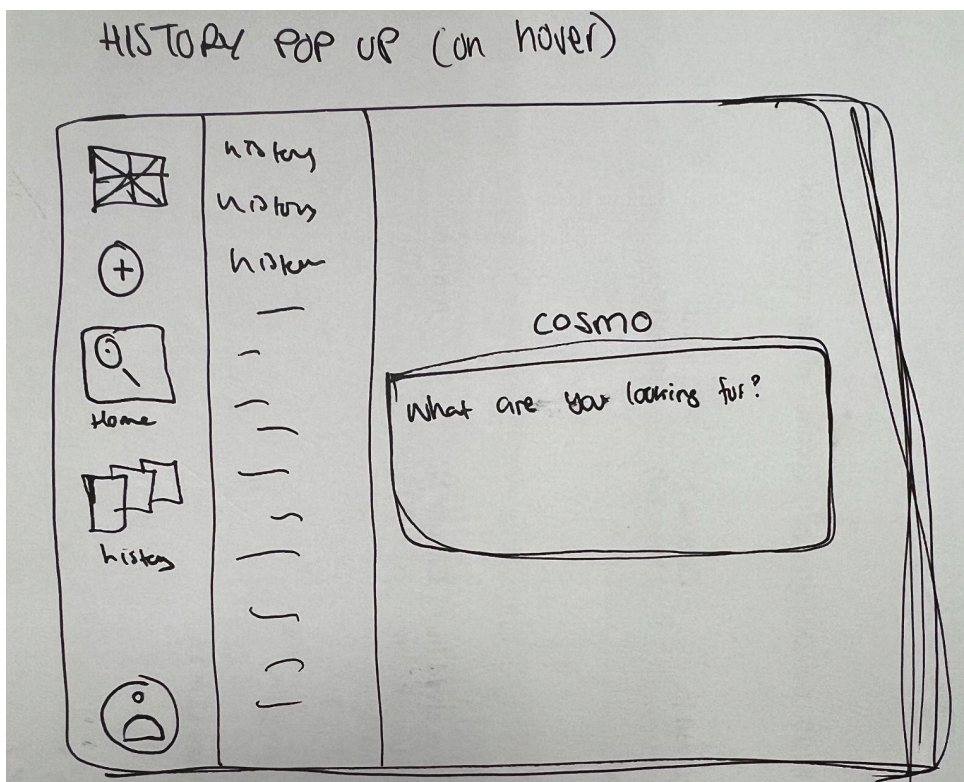


Figure A.3: Low Fidelity Wireframe of History Tab Pop-Up

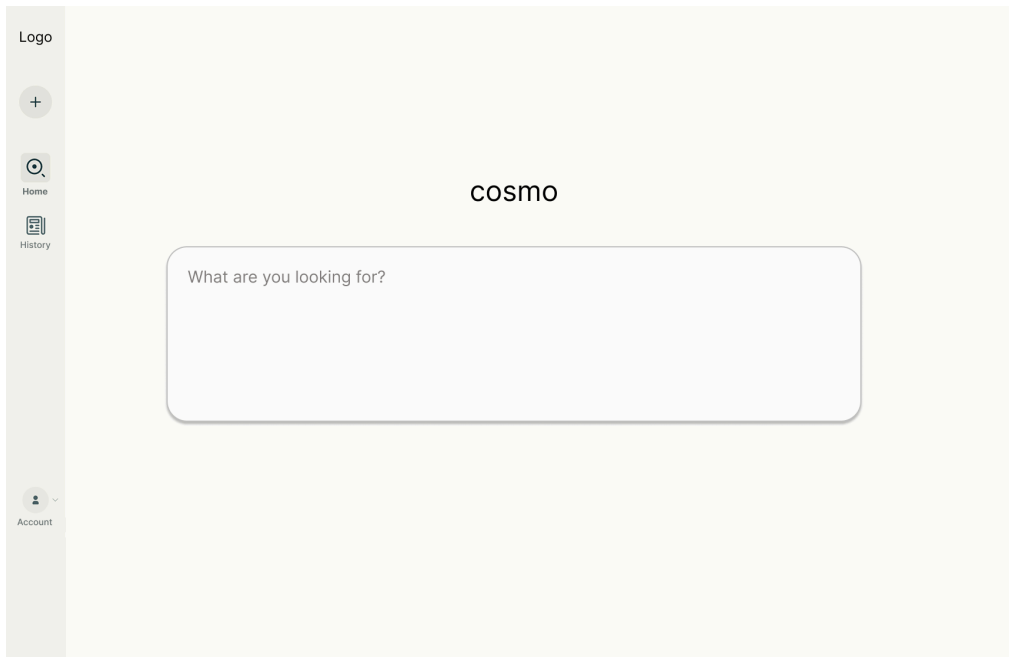


Figure A.4: Medium Fidelity Wireframe of Before Query User Interface

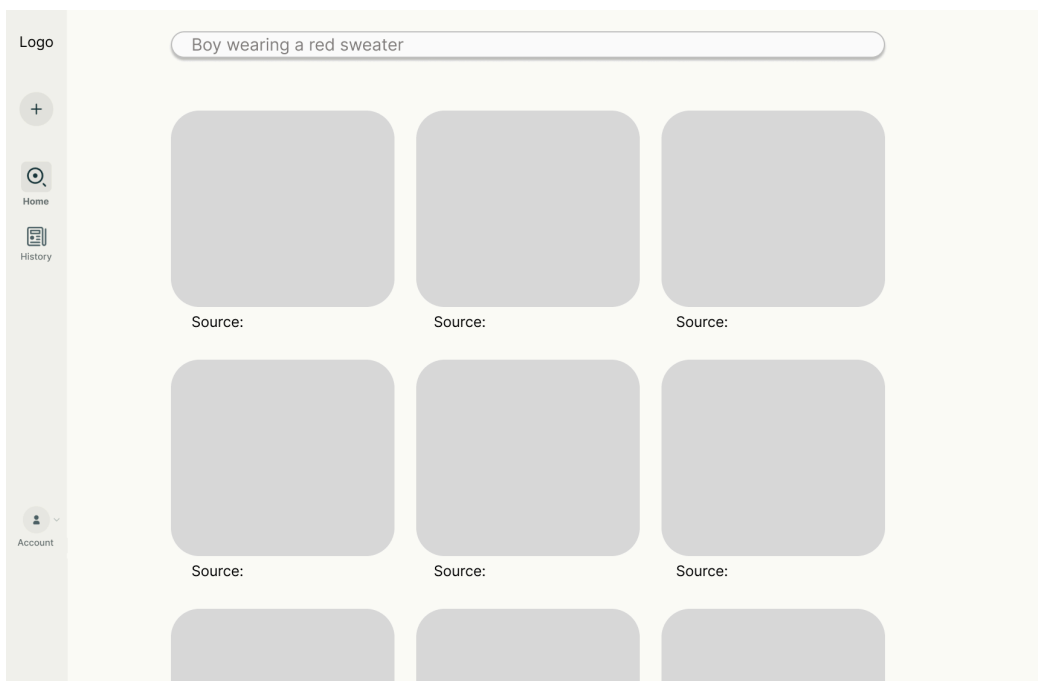


Figure A.5: Medium Fidelity Wireframe of After Query User Interface

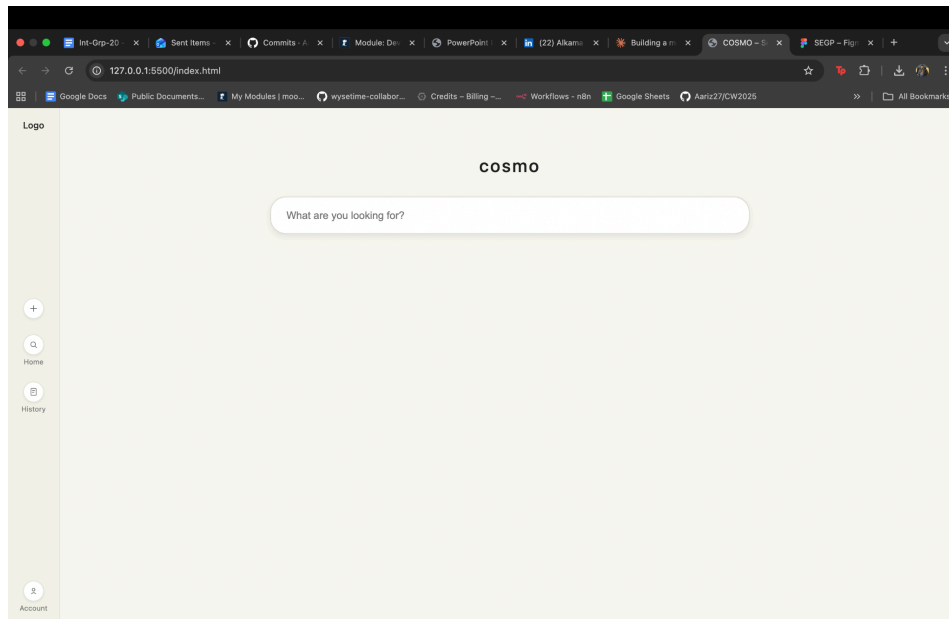


Figure A.6: Initial Streamlit Implementation of Pre-Search User Interface

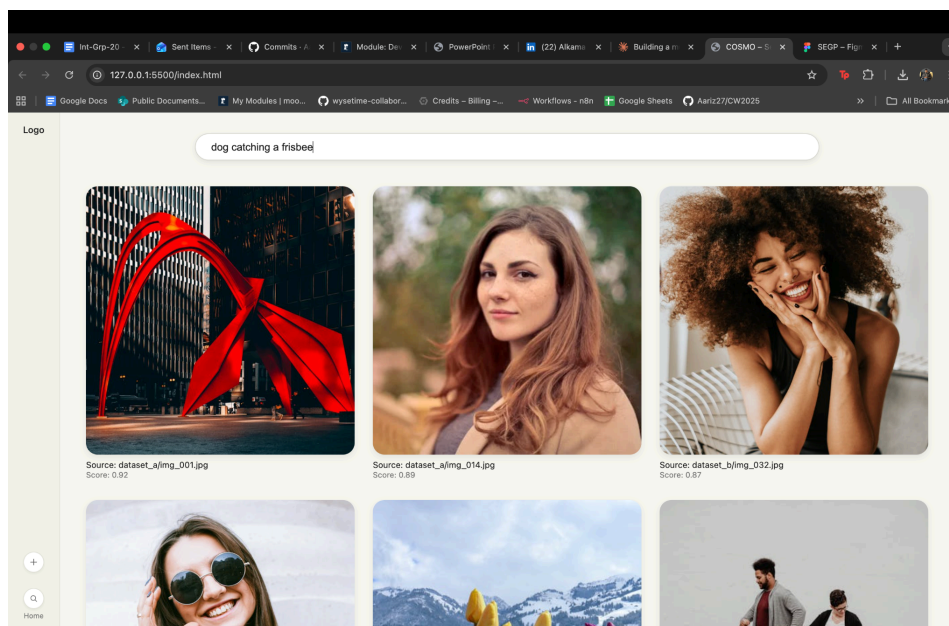


Figure A.7: Initial Implementation of Post-Search User Interface

9.2. Appendix B: Use Case Diagram

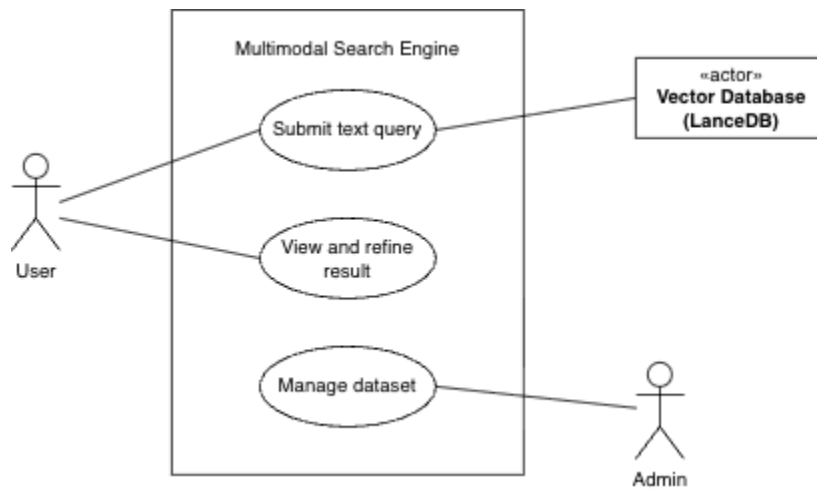


Figure B.1: Use Case Diagram

9.3. Appendix C: High Level Architecture Diagram

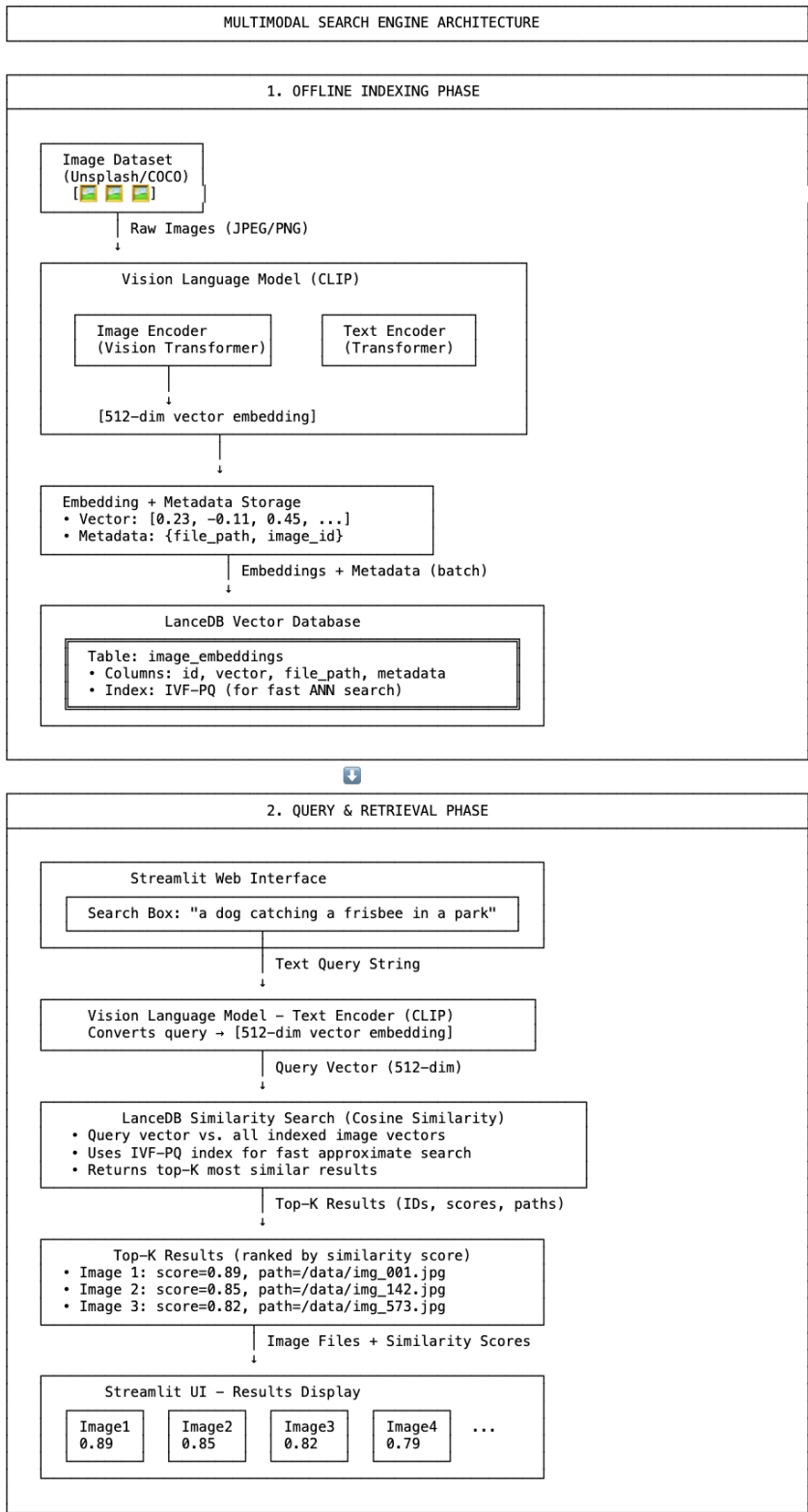


Figure C.1: Conceptual Architecture Diagram

9.4. Appendix D: Gantt Chart

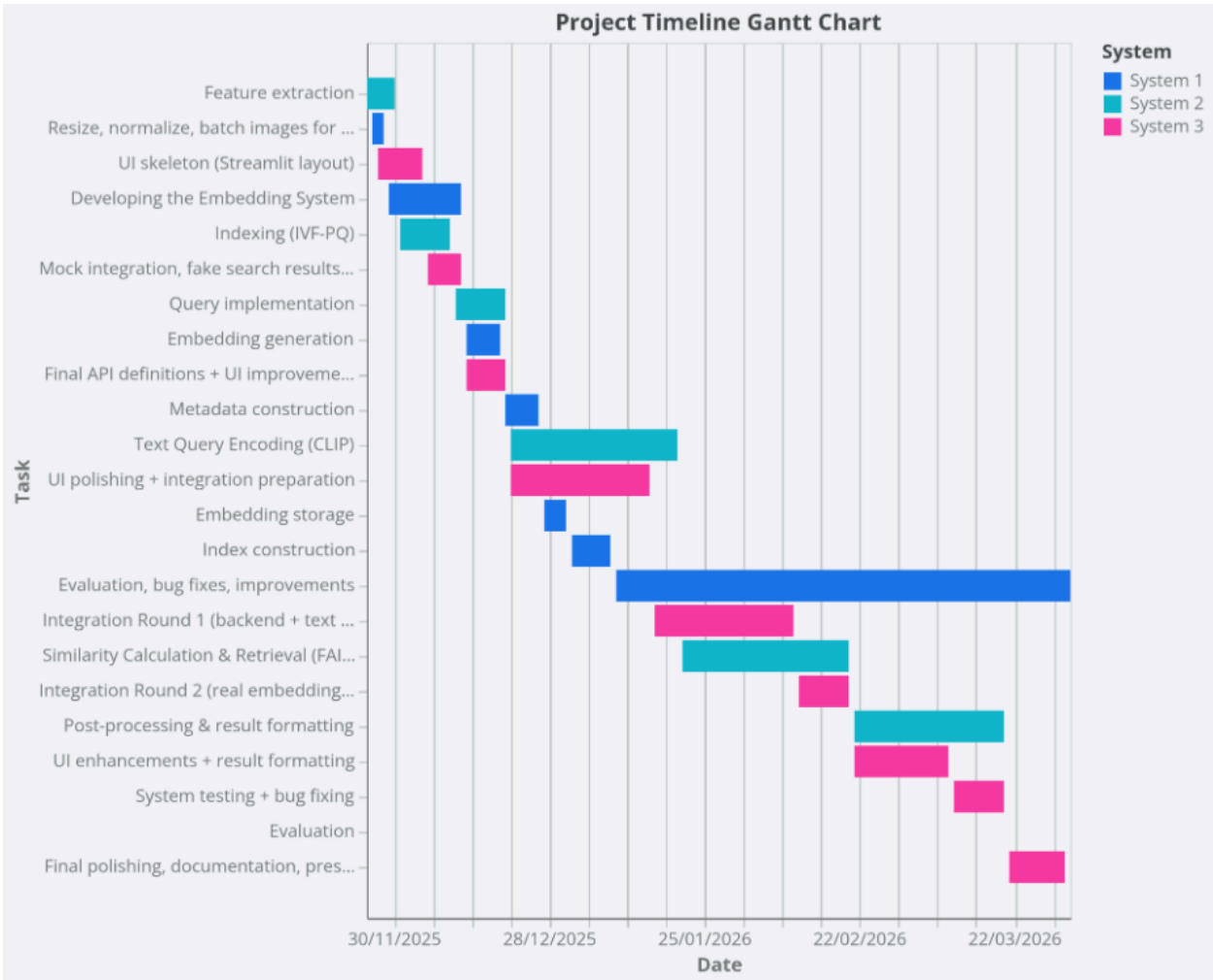


Figure D.1: Gantt Chart of Planned Software Completion

9.5. Appendix E: Meeting Minutes

Meeting	Date	Details
Meeting 1	26/09/2025	The initial goal was defined: individual research on assigned topics to determine how specific skillsets fit into the project.

		Aariz: Deep learning (face recognition models). Karl: Model compression techniques. Song: Embedded systems optimization. Zheng: Database/indexing algorithms. Bessie: C++ or Python programming (language choice, combination, memory cost-effectiveness).
		All research must consider the target environment: devices with limited CPU/GPU and memory resources.
Meeting 2	01/09/2025	The focus was on splitting roles and formulating questions for the professor and the client. The team conducted a research review of the initial findings.
		The team was tasked with finding Anchor papers and deeply researching the technical foundation for the project: how to vectorise images, the use of embedding models, similarity search algorithms, and the impact of the CPU on these three elements.
Meeting 3	11/10/2025	The team met the client, which resulted in a complete change of the project scope.
		The new project is to build an Intelligent Multimodal Search Engine using a Vision Language Model and LanceDB .
Meeting 4	17/11/2025	The team met the client and received specific, applicable advice on improving project management and technical execution.
		Roles should be split based on interest , and everyone's work must align towards the same goal .
		The team was advised to use a Gantt chart and make it date specific to track progress. The client needs to be added to this document.
		Establish biweekly meetings with the client every other Monday at 10:30 am, and institute a daily 5-minute check-in for the group to review progress and challenges.

		Before coding, the team must create a conceptual architecture diagram (a planner diagram) to visualise the data flow and how each layer interacts.
		The question of Who will pay the monthly subscription for the Lance DB? is still unresolved.
Meeting 5	18/11/2025	The team focused on implementing the new advice and structure from the previous meeting.
		The team reviewed and confirmed the conceptual architecture diagram .
		Roles were officially split based on interest: Karl and Song (Ingestion and Offline Indexing), Bessie and Zheng (Querying, Retrieval, and Similarity Search Algorithm), Aariz (UI, WebApp, Integration, and Quality Testing).
		A Gantt chart was created for both the team and the client to track progress.
		The team established and organised the daily 5-minute check-ins .
Meeting 6	02/12/2025	The team conducted a progress review with the client, demonstrating the initial implementation progress across all three subsystems (Offline Indexing, Query/Retrieval, and UI Prototype).
		- The client approved the continued use of the open-source version of LanceDB, resolving the previous outstanding question regarding subscription/cost.
		- Successful demonstration of a working prototype that generated CLIP embeddings, stored them in LanceDB, and retrieved basic results using a text query.
		Established firm deadlines for the next two weeks of development, focusing on full-scale embedding generation and initial tuning of the IVF-PQ index.
Meeting 7	09/12/2025	Group meeting to finalise on UI, technical decisions, map future progress and full runthrough of functioning components.
		Completed about 70% of the interim report.

10. References

DhiWise. (n.d.) *How Google Image Search Works to Surface Accurate Results*. Available at: <https://www.dhiwise.com/post/how-google-image-search-works-to-surface-accurate-result> (Accessed: 15 November 2025)

Frugal Testing. (n.d.) *The Technology Behind Pinterest's Visual Search and AI Discovery*. Available at: <https://www.frugaltesting.com/blog/the-technology-behind-pinterests-visual-search-and-ai-discovery> (Accessed: 21 November 2025).

Google. (n.d.) *How Google Search Works*. Available at: <https://developers.google.com/search/docs/fundamentals/how-search-works> (Accessed: 29 November 2025).

Market Intelo. (2024) *Semantic Search Market Report*. Available at: <https://marketintelo.com/report/semantic-search-market> (Accessed: 3 December 2025).

Rankings.io. (n.d.) *Pinterest as a Search Engine*. Available at: <https://rankings.io/blog/pinterest-as-a-search-engine/> (Accessed: 9 December 2025).

SEOzoom. (n.d.) *How Google Image Search Works*. Available at: <https://www.seozoom.com/google-images/> (Accessed: 10 December 2025).

Shutterstock. (n.d.) *How can I find images*. Available at: <https://www.shutterstock.com/help/en/articles/10617117-how-can-i-find-images> (Accessed: 10 December 2025).

Shutterstock Developers. (n.d.) *Searching (API Documentation)*. Available at: <https://www.shutterstock.com/developers/documentation/searching> (Accessed: 11 December 2025).

Sirv. (n.d.) *How Images are Indexed by Google*. Available at: <https://sirv.com/help/articles/how-images-are-indexed-by-google/> (Accessed: 12 December 2025).